

Cairo University
Faculty of Computers and Artificial Intelligence
Data Science Department

Lumin

“Vision assistance for the visually impaired”

Name	Id
Abdulrahman Gamal Mohammed	20227037
Farida Hamid Mohamed	20227020
Mariam Mohsen Amin	20227023
Salma Abdelhalim Eid	20227014

Under the Supervisors of:

Dr. Ali Zidane

Department of Data Science, Faculty of Computers and Artificial Intelligence,
Cairo University, Egypt

Academic Year

2025-2026

Acknowledgement

First and foremost, we would like to express our deepest gratitude to **Dr. Ali Zidane**, our project supervisor, for his invaluable guidance, continuous encouragement, and constructive feedback throughout every stage of this project. His expertise, insightful advice, and unwavering support played a fundamental role in shaping this work and overcoming the challenges encountered during its development. We are truly grateful for his patience, dedication, and commitment to our learning and success.

We would also like to extend our sincere appreciation to **Dr. Ayman El Kilany**, Head of the Department of Data Science, for his continuous support, encouragement, and for providing an inspiring academic environment that motivated us to pursue this project and enhance our knowledge and skills.

Our heartfelt thanks also go to all the faculty members of the **Faculty of Computer Science and Artificial Intelligence, Cairo University**, for the knowledge and experience they have shared throughout our academic journey.

Abstract

Indoor navigation remains a major challenge for blind and low-vision individuals due to the absence of GPS signals, complex indoor layouts, and the dynamic nature of obstacles within enclosed environments. Although recent technological advances have improved outdoor mobility assistance, effective and accessible indoor solutions are still limited.

This project presents LUMIN, an intelligent cloud-assisted indoor mobility and perception system designed to enhance the independence and safety of visually impaired users in indoor spaces. The system leverages computer vision and deep learning techniques to analyze real-time visual input captured through a standard camera device. Detected objects, obstacles, and environmental cues are processed to generate meaningful auditory feedback that helps users understand their surroundings and navigate safely.

LUMIN adopts a modular architecture that integrates a perception module for object and obstacle detection, a contextual understanding module for indoor scene awareness, and a voice-based interaction module for user communication. To balance accuracy and computational efficiency, the system employs cloud-based processing for intensive tasks while maintaining real-time responsiveness for user interaction.

The proposed system aims to provide an affordable, scalable, and practical indoor mobility solution without requiring specialized hardware or infrastructure. By improving situational awareness and navigation support, LUMIN contributes to the development of inclusive assistive technologies and demonstrates the potential of artificial intelligence in addressing real-world accessibility challenges.

Table of Content

Chapter One

1.1 Background and Motivation.....	9
1.2 Problem Statement.....	9
1.3 Objectives.....	10
1.4 Project Scope and Limitations.....	11
1.5 Project Methodology.....	13

Chapter Two

2.1 Related Works.....	16
2.2 Stakeholders.....	17
2.3 User Analysis.....	17

Chapter Three

3.1 Functional Requirements.....	20
3.2 Non-Functional Requirements.....	21
3.3 UML Diagrams.....	23

Chapter Four

4.1 Introduction.....	26
4.2 Overall System Architecture.....	26
4.3 Backend Architecture.....	27
4.4 Mobile Application Design.....	28
4.5 Voice-First Navigation Flow.....	28
4.6 Roboflow Dataset Collection & Model Fine-Tuning.....	30
4.6.1 Dataset Collection and Annotation.....	30
4.6.2 YOLO Fine-Tuning.....	30
4.7 Object Detection With Depth.....	31
4.8 OCR Block.....	33
4.9 Web Search Block.....	34
4.10 Merging OCR with the Web Search.....	35
4.11 Tool Calling.....	37
4.12 Prompt Engineering.....	39
4.13 Backend API Design.....	41
4.14 Design Decisions.....	42
4.15 Development Environment.....	43

4.16 Chapter Summary.....	44
Chapter Five	
5.1 Introduction.....	46
5.2 Testing Strategy and Tools.....	46
5.2.1 Testing Tools and Technologies.....	46
5.2.2 System Functions Tested.....	47
5.3 Test Cases and Results.....	47
5.3.1 Screenshots of Tested Features.....	51
5.4 Test Results by System Requirement.....	54
5.5 Project Evaluation.....	55
5.5.1 Cost.....	55
5.5.2 Time Compared to Contemporary Systems.....	57
5.5.3 Environmental Impact.....	58
5.5.4 Social and Political Impact.....	58
5.5.5 Limitations Observed During Testing.....	58
5.6 Chapter Summary.....	59
Chapter Six	
6.1 Conclusion.....	61
6.2 Future Work.....	62
Resources.....	64

List of Figures

Figure 1: System Architecture

Figure 2: Use Case Diagram

Figure 3: Implemented System Architecture Diagram

Figure 4: Voice-First Navigation Flow

Figure 5: Object Detection With Depth

Figure 6: Web Search Block

Figure 7: Merging OCR with the Web Search

Figure 8: Tool Calling

Figure 9: Object Detection screen

Figure 10: OCR screen

Figure 11: OCR follow-up web-search fallback

Figure 12: Web Search screen

List of Tables

Table 1.a Detailed Test Cases (Scenario and Input)

Table 1.b Detailed Test Cases (Results and Status)

Table 2: Test Results by Major System Requirement

Table 3: Cost Evaluation

Table 4: Estimated API Usage Cost

Chapter One

Introduction

1.1 Background and Motivation

Independent mobility is essential for personal safety and independence. For blind and low-vision individuals, indoor movement is especially difficult because of limited awareness of surrounding objects and the lack of accessible guidance.

Recent progress in artificial intelligence, particularly in computer vision and speech technologies, has improved the ability of systems to understand environments and interact naturally with users. Cloud computing has further enabled the use of powerful AI models without placing heavy processing demands on user devices.

LUMIN is motivated by these advances and aims to provide a practical indoor assistance system using a smartphone as the main device. The current system combines real-time camera perception, wake-word activation, voice interaction, and an intelligent agent that coordinates user requests and system responses. LUMIN analyzes the current scene and gives short, useful audio feedback that helps the user understand nearby objects, obstacles, and possible movement directions.

The project is also driven by the goal of enhancing accessibility and promoting independent living for individuals with disabilities. By using widely available smartphones and cloud services, LUMIN offers a scalable and affordable solution that supports safety, inclusion, and everyday well-being.

1.2 Problem Statement

Indoor environments are difficult for visually impaired individuals because they lack continuous awareness of what surrounds them. Without visual input, it is hard to know where doors, furniture, corridors, or obstacles are located or how close they are. This limited awareness increases the risk of collisions, confusion, and unsafe movement and reduces confidence when navigating indoors independently.

These challenges affect not only physical safety but also independence and confidence. Uncertainty about the environment often forces visually impaired individuals to rely on memory, assistance from others, or cautious trial-and-error movement, which limits autonomy and restricts daily activities, especially in changing indoor spaces.

Although assistive technologies are available, many do not provide practical, real-time awareness. Some depend on special hardware or prepared environments, while others address only single tasks such as object recognition or text reading. As a result, users still lack a system that continuously informs them about their surroundings clearly and reliably.

This project addresses the need for an accessible smartphone-based system that enhances real-time indoor awareness for visually impaired users. By perceiving the environment, identifying potential hazards, and delivering clear audio feedback, the system aims to improve safety, independence, and overall quality of life in indoor settings.

1.3 Objectives

The primary objective of this project is to design and develop an intelligent indoor assistance system that enhances the safety and independence of blind and low-vision individuals by improving their awareness of surrounding indoor environments. The system aims to support users during indoor movement.

To achieve this goal, we need:

- To develop a smartphone-based assistive system that provides real-time indoor environmental awareness for visually impaired users.
- To support wake-word activation so the user can start interacting with the system without visual or touch-based input.
- To enable natural voice-based interaction through the integration of automatic speech recognition, natural language understanding, and text-to-speech technologies.
- To implement object and obstacle detection capabilities that help users identify nearby elements and potential hazards during movement.
- To incorporate depth and spatial awareness cues that support safer navigation within indoor spaces.
- To incorporate spatial and proximity awareness cues that help users understand whether detected objects are close, far, or positioned to the left, right, or center of the camera view.
- To provide optical character recognition functionality for reading printed text such as labels, signs, and documents.

- To utilize cloud-based processing to support advanced artificial intelligence models while maintaining acceptable response time and reliability.
- To ensure user safety, data privacy, and system reliability through secure communication and controlled data handling.
- To evaluate system performance using key indicators such as response time, detection accuracy, and overall usability.
- To promote user confidence, independence, and safer indoor mobility through continuous situational awareness.
- To integrate a web search agent that can search, retrieve, process, and answer user questions that require online or updated information.

1.4 Project Scope and Limitations

The system will feature the following components:

- **Smartphone-based assistive system:**
The system is implemented as a smartphone application that serves as the primary interface for capturing visual and audio input and delivering auditory feedback to the user.
- **Voice-based interaction:**
The system supports hands-free interaction through automatic speech recognition, natural language understanding, and text-to-speech to enable intuitive communication with the user.
- **Wake-word activation:**
The system supports hands-free activation through a wake-word service, allowing the user to open the assistant without relying on visual or touch-based interaction.
- **Indoor object and obstacle awareness:**
The system detects and identifies indoor objects and potential obstacles that are relevant to user safety and movement within enclosed spaces.
- **Spatial and distance awareness:**
The system provides basic depth and distance cues to help users understand how close objects or obstacles are, supporting safer indoor movement.

- **Text reading functionality:**

Optical character recognition is used to recognize and read printed text such as labels, signs, and documents.

- **User safety focus:**

The system is designed to prioritize clarity, reliability, and minimal mental load during use by using clear instructions.

- **Web search assistance:**

The system includes a web search agent that searches for relevant online information based on the user's question, retrieves useful results, processes the retrieved content, and returns a clear spoken answer.

Despite its capabilities, the project is subject to several limitations:

- **Perception-based assistance only:**

The system provides guidance based on the currently observed environment through the camera and does not support full indoor navigation or long-range path planning across entire buildings.

- **Environmental dependency:**

System accuracy and reliability may be affected by lighting conditions, camera quality, occlusions, and visual clutter commonly found in indoor spaces.

- **Network dependency:**

Although object detection can run locally on the smartphone, web search and some voice services depend on network availability. Weak or unstable connections may affect response time or temporarily limit some features. As the system relies on cloud-based processing, network availability and stability can impact response time and may cause temporary service interruptions.

- **Computational constraints:**

The available computational resources on both the smartphone and cloud infrastructure may limit model complexity, real-time performance, and prolonged continuous usage.

- **Battery consumption:**

Continuous use of the camera, audio processing, and data transmission may lead to increased battery consumption on the smartphone.

- **Evaluation constraints:**
System testing and evaluation are performed in controlled indoor environments, which may not fully represent all real-world indoor conditions and user behaviours.
- **Web information dependency:**
The accuracy of web search responses depends on internet connectivity, search result quality, and the reliability and freshness of online sources.

1.5 Project Methodology

The LUMIN project follows the **Agile project management methodology**, which supports flexible and iterative development. This approach is suitable for projects that involve artificial intelligence and mobile applications, where system behaviour and performance often need to be adjusted during development based on testing results and feedback.

The project adopts the **Scrum framework** to organize the development process. Scrum divides the work into short development cycles called sprints, allowing the team to focus on specific features and evaluate progress regularly. This approach helps manage the complexity of integrating AI-based perception, cloud services, and voice interaction within a smartphone-based system.

Why Agile?

Agile was chosen because it allows gradual development and continuous refinement of system components. Since AI models and user interaction mechanisms require repeated testing and improvement, Agile enables the team to implement features step by step and adjust them as needed before final deployment.

Benefits of Using Agile

- **Flexibility:**
The methodology allows the team to adapt to technical challenges, performance limitations, and evolving system requirements during development.
- **Continuous Improvement:**
Each sprint provides an opportunity to improve key system features such as object detection, voice interaction, and audio feedback through regular testing and evaluation.

- **Stakeholder Engagement:**

Regular reviews help ensure alignment with academic supervisors and user needs, keeping the project focused on safety, usability, and accessibility.

Scrum Framework

Using Scrum, the project is divided into sprints lasting two to three weeks. Each sprint targets specific deliverables, and progress is reviewed at the end of every cycle. Feedback is then incorporated into the following sprint, ensuring steady progress and early identification of issues.

Overall, the Agile methodology enables structured yet flexible development and supports the successful delivery of a reliable and user-focused indoor assistance system.

Chapter Two

Market And Literature Survey

2.1 Related Works

Several smartphone-based systems have been developed to support visually impaired individuals by helping them understand their surroundings. These systems mainly focus on recognizing objects, reading text, or aiding through human support.

One example is **Microsoft Seeing AI**, which uses artificial intelligence to describe objects, text, and scenes captured by the smartphone camera. The application helps users identify items and read printed text, but it mainly provides descriptions and requires the user to actively scan the environment. It does not offer continuous guidance for safe indoor movement.

Another similar system is **Google Lookout**, which provides object recognition and text reading features for Android users. Lookout enhances awareness of nearby elements, focusing on recognition rather than movement assistance. It does not analyze spatial relationships or provide real-time guidance while walking indoors.

Be My Eyes offers a different approach by connecting visually impaired users with sighted volunteers through live video calls. Volunteers help users understand their surroundings or perform specific tasks. While this can be helpful, it depends on human availability, requires a stable internet connection, and may raise privacy concerns. It is also not suitable for continuous, independent use.

ScribeMe is an AI-powered accessibility application designed to assist visually impaired users by converting images of documents, handwritten notes, and printed text into readable digital content. The application focuses mainly on text extraction and transcription, enabling users to access written information through optical character recognition (OCR) and speech output. While ScribeMe is effective for reading and understanding textual content, it does not support object recognition, spatial awareness, or real-time navigation. As a result, it is not suitable for guiding users during indoor movement or helping them avoid obstacles in dynamic environments.

In comparison, **LUMIN** focuses on improving indoor safety and situational awareness through autonomous operation. The system uses smartphone cameras and cloud-based artificial intelligence to analyze the surrounding environment in real time. By providing continuous audio feedback, LUMIN supports safer indoor movement without requiring additional hardware, special infrastructure, or human assistance. This makes LUMIN a more independent and scalable solution for indoor assistance.

2.2 Stakeholders

The LUMIN project involves several stakeholders who are directly or indirectly related to the system.

The **primary stakeholders** are blind and low-vision individuals who will use the system to improve safety and awareness while moving indoors. Their main interest is to receive clear audio feedback that helps them understand their surroundings and avoid obstacles. Ease of use, reliability, and personal safety are the most important concerns for this group.

The **development team** is responsible for designing, implementing, and testing the system. Their role includes building smartphone applications, integrating cloud-based AI services, and ensuring that all system components work correctly and efficiently.

The **secondary stakeholders** are family members or helpers who assist the user during the initial setup of the system. Their role is limited to installing the application, adjusting basic settings such as language and volume, and helping create the user profile. After setup, the system is intended to be used independently.

2.3 User Analysis

Visual impairment represents one of the largest disability groups in Egypt, affecting millions of individuals across different ages and genders. According to the **International Agency for the Prevention of Blindness (IAPB) Vision Atlas (2020)**, an estimated **8.3 million Egyptians** live with some form of vision loss. This figure includes approximately **0.37 million blind individuals, 3.7 million people with moderate to severe vision impairment, 2.2 million with mild impairment, and 2.1 million experiencing near-vision difficulty**. These figures reflect a **medical and epidemiological definition** of vision loss, which includes all clinically measured levels of impairment.

In contrast, **official Egyptian statistics**, such as those reported by the **Central Agency for Public Mobilization and Statistics (CAPMAS)**, focus on **functional disability**, counting individuals whose vision loss significantly affects daily activities and independence. Using this definition, visual impairment remains the most common disability in Egypt, with estimates indicating **approximately 3.5 to 4 million individuals** experiencing serious vision-related functional limitations. This difference highlights the distinction between medical prevalence and practical daily-life impact.

Epidemiological studies from both international and local sources indicate that visual impairment in Egypt is more common among **older adults**, particularly those aged **50 years and**

above, with cataracts and uncorrected refractive errors being the leading causes. Women are slightly more affected than men, reflecting higher rates of avoidable vision loss among older females. Despite many causes being preventable or treatable, the number of visually impaired individuals continues to rise due to population growth and ageing.

From an assistive technology perspective, the most relevant target users for the LUMIN system are **blind individuals and users with moderate to severe low vision**, whose mobility and safety are directly affected by limited environmental awareness. This group represents a substantial population in Egypt and faces daily challenges when navigating indoor environments. These factors confirm the importance and relevance of developing a smartphone-based indoor assistance system tailored to the needs of visually impaired users within the Egyptian context

Chapter Three

System Analysis

3.1 Functional Requirements

The system shall satisfy the following functional requirements:

- **Hands-free voice interaction:**
The system shall enable users to access all core functionalities using voice commands, without requiring visual or touch-based interaction.
- **Smartphone-based data acquisition:**
The system shall capture audio and video data using the smartphone's built-in microphone and camera.
- **Wake-word activation:**
The system shall allow the user to activate the application using a wake word without requiring visual or touch-based interaction.
- **Secure client–cloud communication:**
The system shall securely transmit captured data between the smartphone and cloud services for processing.
- **Voice command routing:**
The system shall recognize the user's spoken intent and route the request to the suitable feature, such as object detection or text reading.
- **Near real-time feedback:**
The system shall process user requests and provide audio feedback within a time frame suitable for safe indoor assistance.
- **Perception-based indoor guidance:**
The system shall provide short-range indoor guidance based on the currently observed environment, such as identifying nearby doors, corridors, room transitions, or visible target objects, without relying on global navigation, mapping, or predefined routes.
- **Scene description generation:**
The system shall convert detected objects into concise scene descriptions that include object names, relative position, and proximity cues.
- **Object and obstacle awareness:**
The system shall detect relevant indoor objects and obstacles and notify the user through clear audio alerts.

- **Distance and proximity awareness:**
The system shall estimate relative distances to detected objects or obstacles and communicate proximity information to support safer movement.
- **Context-aware assistance:**
The system shall adapt its feedback based on the user's context and intent, such as moving, searching for an object, or requesting information.
- **Text recognition and reading (OCR):**
The system shall support reading printed or labelled text on demand and deliver the content using speech output.
- **Continuous interaction flow:**
The system shall support continuous interaction during use without requiring repeated manual activation for each request.
- **Service coordination:**
The system shall coordinate mobile sensing, local object detection, speech recognition, web search, answer generation, and text-to-speech output during operation.
- **Privacy-aware operation:**
The system shall protect user privacy by encrypting transmitted data and minimizing the storage of sensitive audio or visual information.
- **Web search support:**
The system shall support user questions that require online information by searching the web, retrieving relevant results, processing the retrieved content, and delivering the answer through speech.

3.2 Non-Functional Requirements

- **Performance and responsiveness:**
The system shall provide timely audio feedback suitable for safe indoor assistance, ensuring that delays do not negatively affect user awareness or decision-making.
- **Reliability:**
The system shall operate consistently during normal use and handle temporary failures (such as network delays or service interruptions) without causing abrupt or unsafe behaviour.

- **Usability:**
The system shall be easy to use by blind and low-vision individuals, requiring minimal learning effort and supporting clear, concise audio feedback.
- **Accessibility:**
The system shall comply with accessibility principles by supporting voice-only interaction, adjustable speech settings, and minimal cognitive load.
- **Scalability:**
The system shall be designed to support an increasing number of users and future feature expansion without major architectural changes.
- **Security:**
The system shall protect user data by using secure communication channels and prevent unauthorized access to system services.
- **Privacy:**
The system shall minimize the collection and storage of sensitive audio and visual data and process user information responsibly.
- **Availability:**
The system shall aim for high availability during normal operation, while clearly handling cases where cloud services are temporarily unavailable.
- **Maintainability:**
The system shall be modular and well-structured to allow future updates, bug fixes, and model improvements with minimal disruption.
- **Portability:**
The system shall be deployable on common smartphone platforms without requiring specialized hardware or device modifications.
- **Energy efficiency:**
The system shall be designed to limit battery consumption on the smartphone during continuous use.
- **Adaptability:**
The system shall be capable of adapting to different indoor environments and user preferences without requiring manual reconfiguration.
- **Extensibility:**
The system architecture shall allow the integration of additional services or

improvements, such as enhanced perception models or new interaction modes, in future versions.

- **Fault tolerance:**

The system shall detect service interruptions or communication failures and respond gracefully to maintain usability whenever possible.

3.3 UML Diagrams

System Architecture:

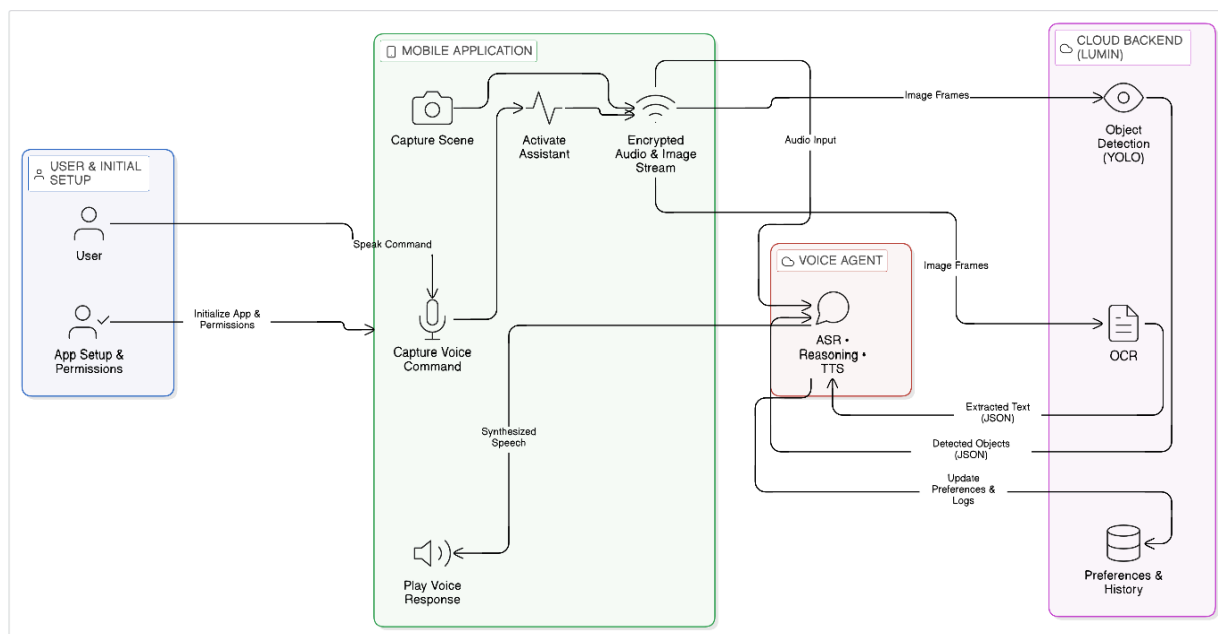


Fig.1 System Architecture

The system architecture presents the workflow of LUMIN, where the mobile application captures voice commands and camera input from the user. The application performs local object detection using YOLO-based models and converts the detections into scene context, including object names, positions, and proximity cues. If the user asks a question that requires online or updated information, the web search agent searches the web, retrieves relevant results, processes the retrieved content, and generates a clear answer. The final response is returned to the user as spoken feedback.

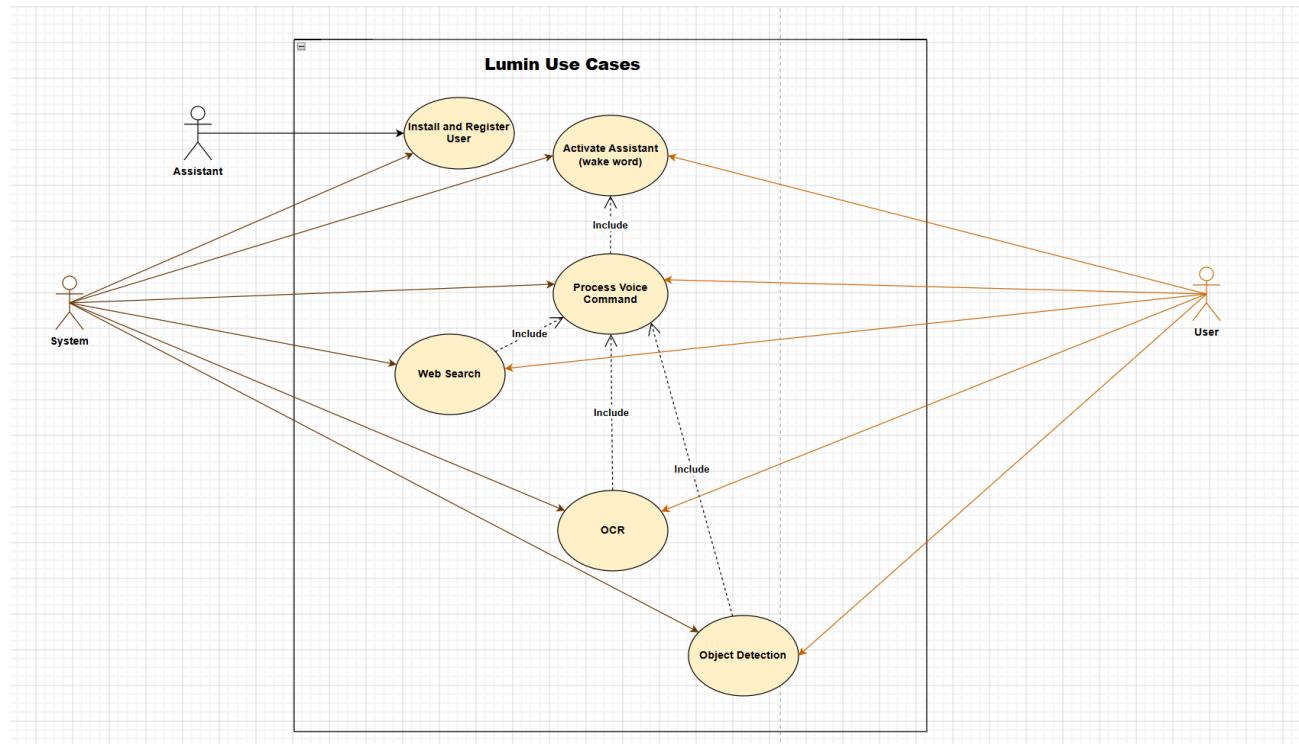


Fig. 2 Use Case Diagram

The use case diagram illustrates the main interactions between the visually impaired user, the assistant, and the LUMIN system. The user can activate the system using a wake word, issue voice commands, ask about the surrounding environment, request object detection, request text reading, and ask questions that require web search. The system processes the request through the mobile application, local perception models, backend services, and web search agent when needed, then responds using audio feedback. The assistant may help during installation, registration, and initial setup.

Chapter Four

Design And Implementation

4.1 Introduction

The implementation of Lumin was designed around one simple story: a visually impaired user says “Lumin”; the phone wakes up, listens, understands what the user needs, and opens the correct assistance mode. From there, the application becomes a voice-guided companion that can describe objects, estimate relative depth, read text, or search the web.

Lumin is not only a mobile interface. It is a connected system made of several working blocks: a Flutter mobile application, an Android native wake-word and audio layer, local computer vision models, a FastAPI backend, OpenAI language and speech models, Google Cloud Vision OCR, and a prompt system that controls how the assistant speaks and reasons.

4.2 Overall System Architecture

Lumin follows a client-backend architecture. The Flutter app runs on the mobile device and handles the camera, user interface, local object detection, native audio recording, and text-to-speech playback. The backend keeps API keys and connects the app to OpenAI, Google Cloud Vision, web search, transcription, text-to-speech, and depth estimation.

The main components are:

- **Flutter mobile app:** user interface, camera screens, voice flow, navigation, local YOLO inference.
- **Android native layer:** wake-word foreground service, microphone recording, WAV generation, native Arabic TTS, beep feedback.
- **FastAPI backend:** central API layer for AI requests and model services.
- **OpenAI services:** text reasoning, transcription, web search, and TTS.
- **Google Cloud Vision:** OCR text extraction.
- **Local/edge models:** YOLO object detection and TFLite MiDaS depth estimation.

System Architecture Diagram:

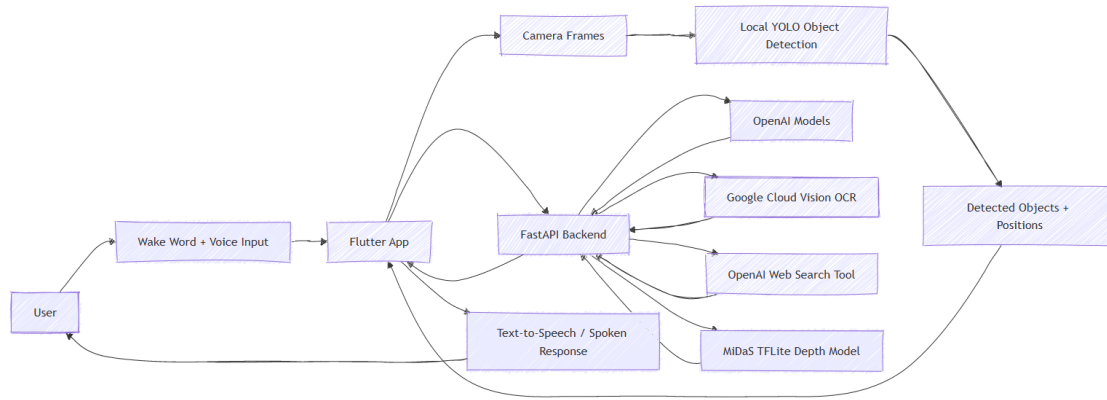


Fig. 3

4.3 Backend Architecture

LUMIN follows a hybrid client-server mobile architecture. The Flutter mobile application acts as the client, while the FastAPI backend acts as the central server responsible for coordinating AI services and protecting external API keys.

During development and testing, the Flutter application runs on a real Android mobile phone, while the FastAPI backend runs locally on the developer machine. The phone communicates with the backend through the machine's local network IP address, so both the phone and the backend machine must be connected to the same network. This setup allows the system to be tested under realistic mobile conditions using the phone camera, microphone, speaker, and on-device object detection, while keeping sensitive API keys inside the backend environment.

The backend is responsible for handling secure communication with OpenAI and Google Cloud Vision services. It also manages transcription, OCR processing, prompt orchestration, web search, text-to-speech generation, and MiDaS depth estimation. By placing these responsibilities in the backend, the Flutter application remains lightweight and doesn't contain complex AI service logic.

The backend receives requests from the Flutter app through HTTP endpoints. Each endpoint performs a specific task, such as converting speech to text, routing commands, reading text

from images, answering OCR follow-up questions, performing web search, estimating depth, or generating speech output.

The system therefore combines edge processing and external AI services. Lightweight real-time operations, such as camera access, native audio recording, and YOLO object detection, are performed on the mobile device. More computationally intensive or security-sensitive tasks, such as transcription, OCR processing, LLM reasoning, web search, and TTS, are handled by the backend and connected external AI services.

4.4 Mobile Application Design

The mobile application was implemented using Flutter. The app version is 1.0.0+1, with Dart SDK (3.10.8). The design uses a dark theme, large buttons, high-contrast panels, and simple icons. This is suitable for an accessibility-focused application because it reduces visual clutter and supports users who may still have partial vision.

The app contains the following main screens:

- **Onboarding / Setup Screen:** asks for initial permissions and marks setup as complete.
- **Home Screen:** listens for the user's command and routes it to the right feature.
- **Object Detection Screen:** opens the camera and detects nearby objects.
- **OCR Screen:** captures a camera frame and reads visible text.
- **Web Search Screen:** lets the user ask current-information questions.
- **Profile Screen:** stores basic user details and emergency contact information.

The design is voice-first. The Home screen automatically listens for a short command, sends it to the backend for transcription, then routes the user to the correct mode.

4.5 Voice-First Navigation Flow

The main user journey begins with the wake word or setting it using Bixby. On Android, a foreground service uses Picovoice Porcupine with the keyword file `lumen_en_android_v4_0_0.ppn`. When the service detects the wake word, it launches the app and sends an event to Flutter through a method channel.

After the app opens, the Home screen records about four seconds of audio using the native Android microphone layer. The audio is converted into WAV format, base64 encoded, and sent to the backend `/transcribe` endpoint. The backend uses the OpenAI transcription model, default `gpt-4o-mini-transcribe`, and returns text.

The transcribed command is then sent to `/route-home-command`. The backend uses OpenAI function calling to choose one of the available tools:

- `open_object_detection`
- `open_ocr`
- `open_web_search`
- `clarify` (when the spoken command is too ambiguous to route safely. Instead of opening the wrong feature, the backend returns a short clarification question)

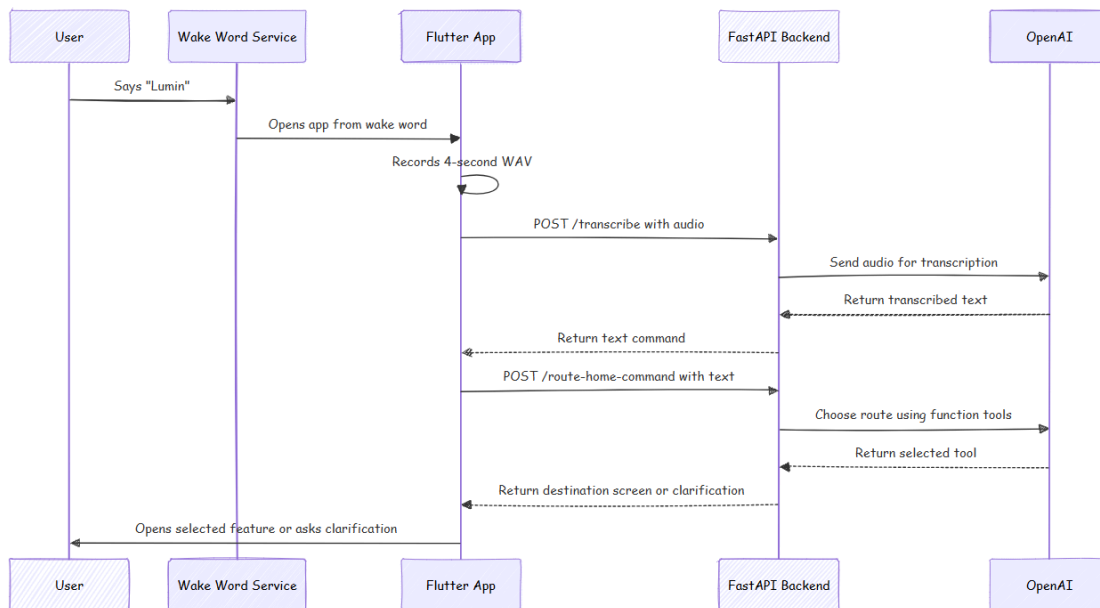


Fig. 4

4.6 Roboflow Dataset Collection & Model Fine-Tuning

4.6.1 Dataset Collection and Annotation

To improve object detection performance in indoor environments, a custom dataset was created and managed using **Roboflow**. The purpose of the dataset was not only to increase the number of detectable objects but also to ensure that the training data closely represented the environments in which LUMIN is expected to operate.

Since the target users are primarily visually impaired individuals living in Egyptian homes, the dataset focused on objects commonly found in Egyptian indoor environments. Images were collected and annotated to include furniture, doors, tables, chairs, sofas, cabinets, and frequently encountered household items such as drawers, door handles, electrical sockets, pillows, fans, and washing machines.

A total of **26 custom indoor object classes** were added to complement the **80 object categories provided by the standard COCO dataset**. This combination enables the system to recognize both general everyday objects and indoor-specific objects that are particularly important for navigation and environmental awareness.

Roboflow was used to organize the dataset, perform image annotation, manage dataset versions, and apply preprocessing and augmentation techniques before training. Data augmentation included operations such as resizing, flipping, rotation, brightness adjustment, and scaling to improve model robustness under varying camera angles and lighting conditions.



4.6.2 YOLO Fine-Tuning

After preparing the dataset, the object detection model was fine-tuned using the **YOLO26** architecture. Rather than training a model from scratch, transfer learning was employed by initializing the model with pretrained weights and then retraining it using the custom Roboflow dataset.

Fine-tuning allows the model to retain the general visual knowledge learned from large-scale datasets while adapting to the specific indoor objects required by the LUMIN application. During training, the model learned to distinguish the newly introduced household objects.

The latest **YOLO26** architecture was selected because it offers improvements in detection accuracy, inference efficiency, and real-time performance compared with earlier YOLO generations. These characteristics make it particularly suitable for assistive mobile applications, where both low latency and high detection accuracy are essential.

After training, the best-performing model was exported as a TensorFlow Lite model (`lumin_best_float32.tflite`) and integrated into the Flutter application. During runtime, this custom model operates alongside the standard COCO-based YOLO model. The outputs of both models are merged to provide comprehensive scene understanding, allowing LUMIN to recognize both common everyday objects and the additional indoor-specific classes introduced during fine-tuning.



4.7 Object Detection With Depth

The object detection block is implemented in the `ObjectDetectionScreen`. It uses the phone camera through `YOLOView` and performs local inference using TFLite YOLO models.

Two detection sources are used:

- **COCO/general object model:** `yolo26n_float32_final`
- **Custom indoor model:** `lumin_best_float32`

The general model detects common objects, while the custom model is intended to improve detection for indoor objects that are more relevant to the Lumin use case. The app combines detections from both models and displays bounding boxes with labels and confidence values.

To make object detection more useful for a visually impaired user, detections are converted into simple scene descriptions. For example:

- “Chair at center, close”
- “Door at left, far”
- “Table at right, very close”

Depth estimation is implemented using a MiDaS TFLite model on the backend. After YOLO detects objects in the camera frame, the Flutter app sends the full image and the normalized bounding boxes of the detected objects to the `/depth/midas` endpoint. The backend loads the `midas_v2_1_small.tflite` model and resizes the image to the model input size, which is 256 by default. The image is then passed through the MiDaS model to generate a depth map. Since MiDaS produces relative depth rather than exact physical distance, the backend normalizes the depth map into a 0-255 range. For each detected object, the backend samples a small patch around the center of its bounding box and calculates the median depth score. This score is then converted into a proximity label such as VERY CLOSE, CLOSE, MID, or FAR. These labels are sent back to Flutter and added to the object description used by the assistant.

This depth is not treated as an exact distance. It is used as a relative safety hint. The assistant is instructed not to say exact measurements unless a tool provides an exact measurement.

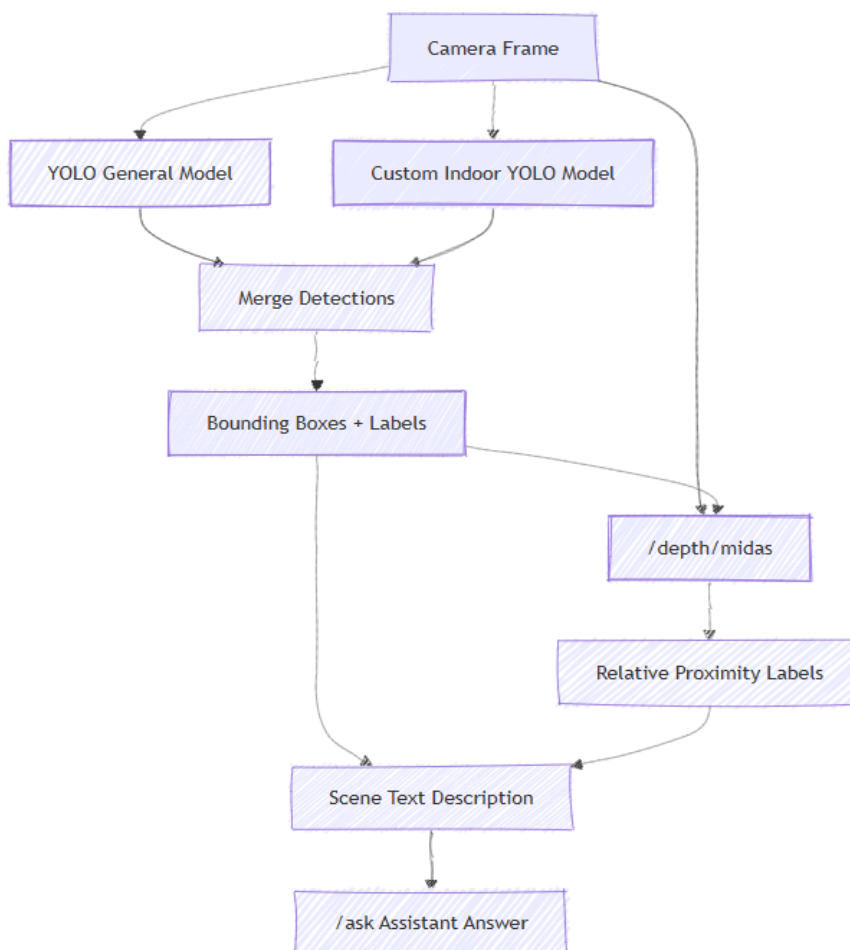


Fig. 5

4.8 OCR Block

The OCR block is implemented in the OCRScreen. It is designed for reading documents, labels, signs, packages, pages, and screens. The screen opens the camera, captures a frame, and sends the image to the backend /read-text endpoint.

The backend uses Google Cloud Vision API with text detection. After OCR extraction, the raw text is not simply returned to the user. Instead, it is processed by OpenAI using the OCR prompt. This allows Lumin to speak the content clearly, summarize when needed, and preserve the original language of the text.

The OCR flow also supports follow-up questions. After Lumin reads the text, it listens again. The user can ask questions such as:

- “What is the expiry date?”
- “Read the next page.”
- “What is this product?”

For follow-up questions, the app sends the question and stored OCR text to /read-text/question. The backend first checks whether the answer can be produced from the captured text. If not, it can route the question to a web search.



4.9 Web Search Block

The web search block is implemented in WebSearchScreen. It is used when the user asks for current or external information. The app records the spoken question, sends it to /transcribe, then sends the text question to /web-search.

The backend uses OpenAI's web search tool through the Responses API. It also extracts URL citations and returns them to the Flutter app. The spoken answer is cleaned and summarized so it sounds natural when read aloud. The app displays the answer and shows source buttons that can open the cited web pages.

The screen keeps a short conversation history, so follow-up questions can refer to the previous web search context.

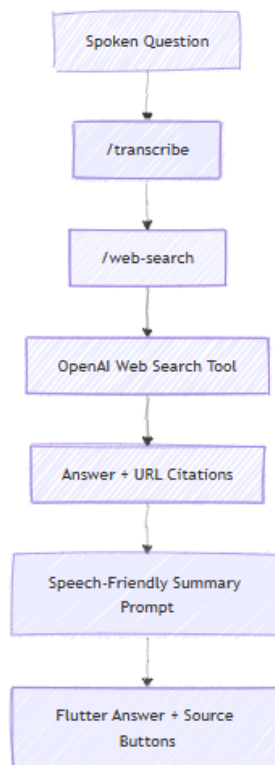


Fig. 6

4.10 Merging OCR with the Web Search

After both blocks were working separately, OCR was integrated with the agent to create a more intelligent reading experience. Instead of only reading extracted text, the OCR module now sends the captured text to the agent and allows the user to ask follow-up questions.

This merge changed OCR from a simple text reader into an interactive text assistant.

The merged flow works as follows:

1. The app captures an image from the camera.
2. Google Cloud Vision extracts the text.
3. The OCR prompt prepares a spoken reading or summary.
4. The app speaks the result.
5. The user can ask a follow-up question.
6. The backend sends the question and OCR text to the OCR context router.
7. The router decides whether the answer can be produced from the captured text or whether a web search is needed.
8. If the text is enough, the agent answers based on the OCR context.
9. If the text is not enough, the system combines OCR context with web search.
10. The final answer is generated by the LLM and spoken to the user.

Integrating OCR with the AI agent transforms the reading process from simple text extraction into context-aware conversational assistance. The system no longer only extracts text; it reasons over the extracted text, answers follow-up questions, and calls a web search when the captured text alone is not enough.

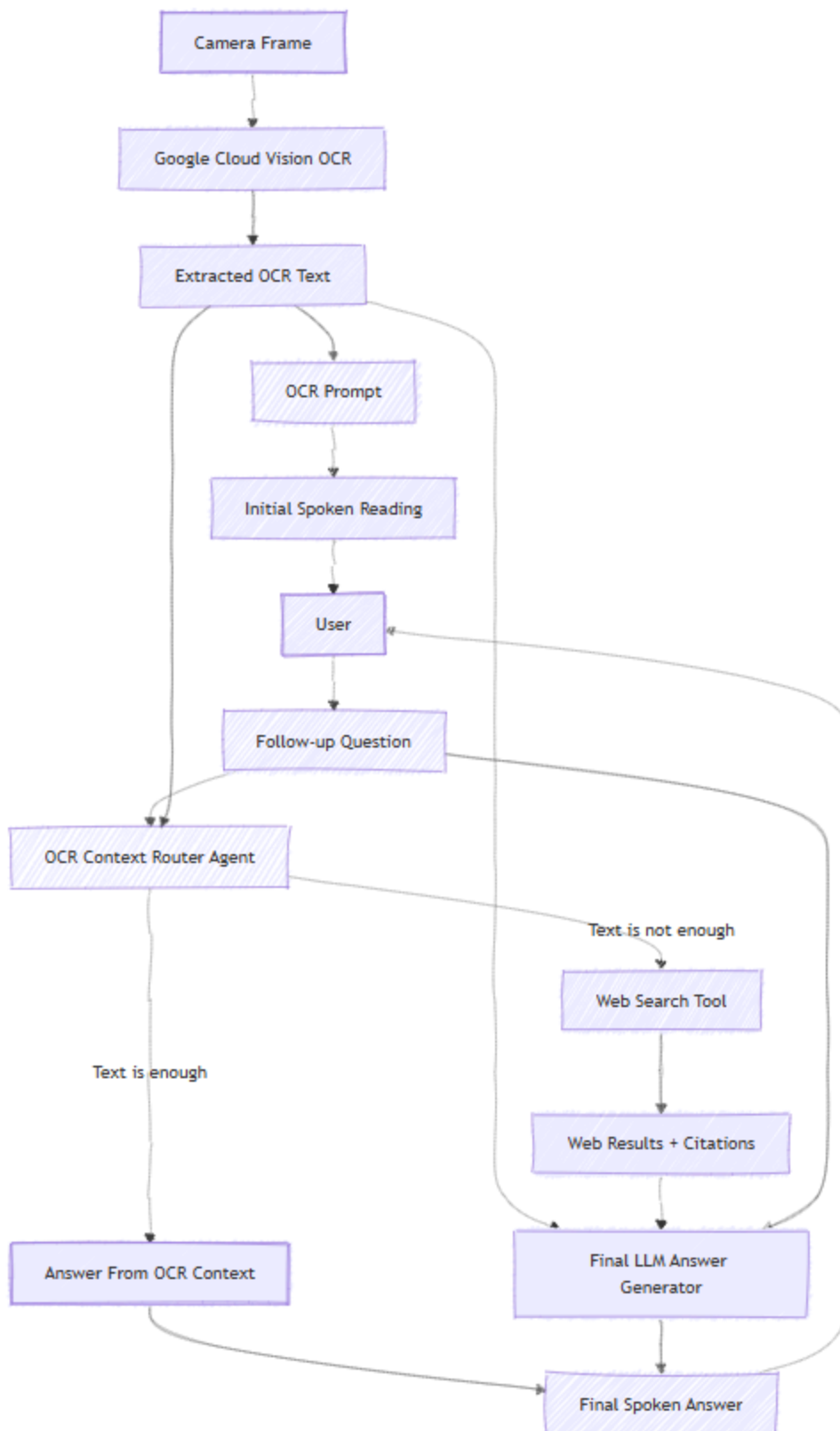


Fig. 7

4.11 Tool Calling

What is Tool Calling?

Traditional software applications rely on manually written conditional statements to determine which function should be executed in response to a user's request. As the number of supported features increases, maintaining these rule-based systems becomes increasingly difficult and inflexible.

Modern large language models provide an alternative approach through **tool calling**, which enables the model to analyze the user's intent and select the most appropriate predefined function automatically. Instead of directly generating an answer, the language model first determines which tool is required, invokes that tool, and then incorporates its output into the final response. This approach allows different AI services to operate together while maintaining a natural conversational interface.

Tool Calling in LUMIN

LUMIN uses OpenAI tool calling to coordinate multiple application capabilities through a single conversational interface. Rather than requiring the user to manually select different application screens, the AI agent determines the appropriate action based on the spoken request.

The first use of tool calling occurs immediately after speech transcription. Once the user's voice command has been converted into text, it is sent to the `/route-home-command` endpoint. The language model analyzes the command and selects one of the available application functions.

The routing process can return one of the following actions:

- **Open object detection** – Opens the object detection module for scene understanding and obstacle awareness.
- **Open OCR** – Opens the OCR module to read printed text.
- **Open web search** – Opens the web search module for questions requiring external knowledge.
- **clarify** – Requests additional information whenever the user's intent cannot be determined with sufficient confidence.

For example, if the user says *"Read this document,"* the model selects the OCR tool. If the user asks *"What is in front of me?"*, the object detection module is selected. Similarly, questions such as *"Who invented the telephone?"* are automatically routed to the web search module.

This dynamic routing eliminates the need for manually written decision trees and allows the application to remain easily extensible as additional capabilities are introduced.

Tool Calling During OCR

Tool calling is also used after OCR processing. Once the text has been extracted from an image, users may ask follow-up questions about the captured content.

Instead of assuming that every question can be answered from the recognized text, the backend evaluates whether the OCR context contains sufficient information. If the answer exists within the extracted document, the AI agent generates the response directly from the OCR context. Otherwise, the system automatically invokes the web search tool to retrieve additional information before generating the final spoken response.

This mechanism enables natural conversations without requiring the user to switch between application features manually.

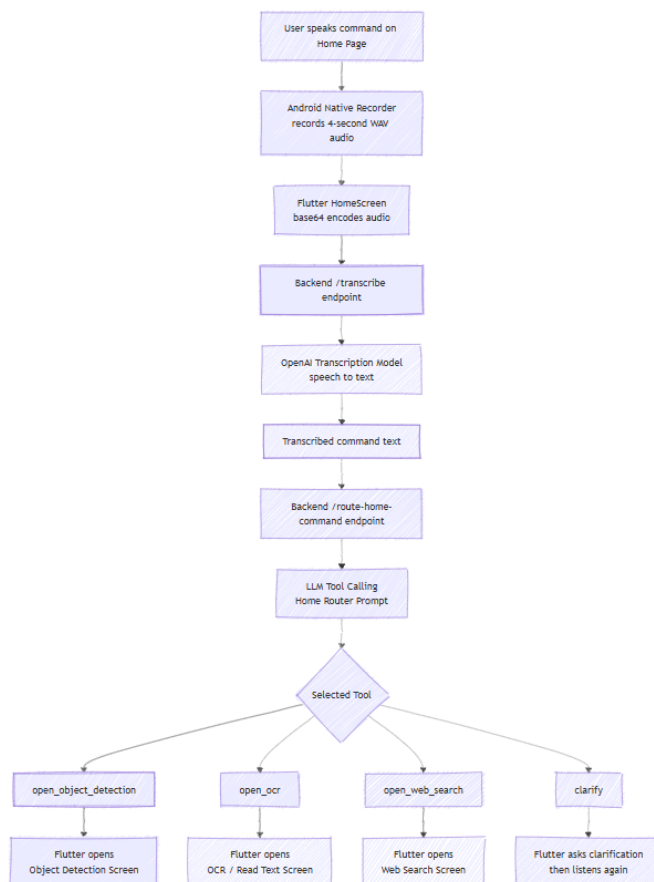


Fig. 8

4.12 Prompt Engineering

Prompt Engineering

Prompt engineering is the process of designing structured instructions that guide the behavior of a large language model toward producing accurate, consistent, and task-specific responses. Rather than relying on generic prompts, LUMIN employs specialized prompt templates tailored to each system component. These prompts define the assistant's behavior, response style, safety constraints, and task-specific instructions, ensuring that responses remain reliable and appropriate for visually impaired users.

The prompts are stored as separate Markdown files within the backend project and are loaded dynamically during runtime. A prompt loader replaces predefined variables before the prompt is sent to the language model. This modular design simplifies maintenance and allows prompts to be modified without changing the application code.

System Prompt

The system prompt defines the overall behavior of the LUMIN assistant. It establishes the assistant as an indoor navigation and accessibility assistant designed for blind and low-vision users.

The system prompt specifies several important constraints, including:

- Respond primarily in Egyptian Arabic.
- Keep spoken responses short and easy to understand.
- Never describe objects that were not detected.
- Avoid reporting exact distances unless reliable measurements are available.
- Prioritize user safety whenever navigation advice is provided.

These instructions ensure consistent behavior regardless of which application module is being used.

Task-Specific Prompts

In addition to the system prompt, LUMIN uses specialized prompts for different application features.

Object Detection Prompt

The object detection prompt instructs the language model to generate concise descriptions of detected objects using their labels, relative positions, and proximity information. The prompt also prevents the model from inventing objects or providing unsupported navigation instructions.

OCR Prompt

The OCR prompt transforms raw OCR output into speech-friendly responses. Instead of reading extracted text verbatim, the prompt allows the model to summarize long passages, ignore unnecessary formatting, and preserve the original language of the recognized text whenever appropriate.

OCR Question Prompt

This prompt is used after the initial OCR reading. It instructs the model to answer follow-up questions using the previously extracted OCR context before considering any external information sources.

Web Search Prompt

The web search prompt guides the language model in generating search queries, summarizing retrieved information, and producing concise spoken responses while preserving source citations.

Speech Summarization Prompt

After web search results are obtained, a dedicated summarization prompt converts the retrieved information into short, natural sentences that are suitable for text-to-speech output. This avoids unnecessarily long spoken responses and improves the listening experience.

Safety Prompts

LUMIN also includes safety-oriented prompts that prevent misleading responses. These prompts instruct the language model to avoid making unsupported claims about the environment, refrain from providing exact distances without reliable measurements, and avoid stating that a path is safe unless sufficient evidence is available. These constraints are particularly important because the application is intended to assist visually impaired users during indoor navigation.

4.13 Backend API Design

The FastAPI backend exposes a group of HTTP endpoints that allow the Flutter application to access AI services without directly communicating with third-party providers. This design improves security, simplifies the mobile application, and centralizes AI orchestration inside the backend.

EndPoint	Method	Purpose
/health	GET	Checks backend availability and configured providers.
/transcribe	POST	Converts recorded user audio into text using OpenAI transcription.
/route-home-command	POST	Routes the transcribed home command to the correct application feature using tool calling.
/ask	POST	Generates an assistant answer based on detected objects and scene description.
/read-text	POST	Sends an image to Google Cloud Vision OCR and generates an initial spoken reading.

/read-text/question	POST	Answers follow-up questions using OCR context, with web search fallback when needed.
/web-search	POST	Performs a web search and returns a summarized spoken answer with citations.
/depth/midas	POST	Estimates relative depth for detected object bounding boxes using MiDaS.
/tts-openai	POST	Generates spoken audio using OpenAI TTS.
/tts-edge	POST	Provides fallback speech generation using Edge TTS.

4.14 Design Decisions

Several design decisions were made to balance usability, performance, and accessibility.

First, LUMIN uses a hybrid client-server architecture instead of processing everything on the mobile device. This allows fast tasks such as YOLO object detection to run locally, while heavier tasks such as OCR, transcription, reasoning, and web search are handled by the backend and external AI services.

Second, object detection is performed using two YOLO models. The general COCO-based model provides broad object coverage, while the custom indoor model improves detection of household objects that are more relevant to visually impaired users in indoor environments.

Third, MiDaS depth estimation is used only as a relative proximity indicator, not as an exact measurement tool. This decision improves safety because the assistant avoids giving precise distance values that may be unreliable.

Fourth, OCR was integrated with the AI agent instead of returning raw extracted text directly. This allows the system to summarize long text, preserve context, and answer follow-up questions naturally.

Finally, tool calling was used instead of fixed keyword rules. This makes the system more flexible because the AI agent can understand different ways of asking for the same feature and route the user to the correct module.

4.15 Development Environment

During development, LUMIN was deployed using a local client-server setup. The Flutter application was installed and tested on a real Android device, while the FastAPI backend server ran on the developer machine.

The backend was started locally using Uvicorn on port 3000. The mobile application connected to it using the developer machine's local network IP address. This required both the phone and the backend machine to be connected to the same Wi-Fi network.

This setup was useful because it allowed the team to test real mobile features, including camera access, microphone recording, speaker playback, wake-word detection, local YOLO inference, and screen navigation. At the same time, it kept API keys and AI service integrations inside the backend, rather than exposing them in the mobile application source code.

In future deployment, the backend can be moved from a local development machine to a cloud server. This would allow the mobile application to access the backend from any network, improve availability, and support multiple users.

4.16 Chapter Summary

This chapter presented the design and implementation of the LUMIN system. It explained the hybrid client-server architecture, the Flutter mobile application, the FastAPI backend, voice-first navigation, custom YOLO fine-tuning, object detection with relative depth estimation, OCR, web search, OCR-agent integration, tool calling, and prompt engineering.

The implementation demonstrates how multiple AI services can be combined into a single voice-first assistive system. Instead of treating object detection, OCR, and web search as isolated features, LUMIN connects them through an AI agent that reasons over user commands, selects the appropriate tools, and generates short spoken responses suitable for blind and low-vision users.

Chapter Five

Testing And Evaluation

5.1 Introduction

This chapter presents the testing and evaluation process carried out on Lumin, the Flutter-based mobile assistive application supported by a FastAPI backend. The purpose of testing was to verify that Lumin's core assistive functions: voice-first navigation, object detection with depth estimation, optical character recognition (OCR), web search, and speech interaction behave correctly under realistic usage conditions, and to identify any limitations before the system is considered complete. The purpose of evaluation is to assess Lumin more broadly, beyond individual pass/fail outcomes, in terms of cost, performance relative to comparable systems, and the environmental, social, and political implications of the system.

5.2 Testing Strategy and Tools

Testing was conducted iteratively throughout development and consolidated into a final verification pass. Several types of testing were applied, each targeting a different aspect of Lumin's behavior:

- **Functional testing** : verifying that individual features (wake-word activation, object detection, OCR, web search, etc.) behave as intended.
- **Integration testing** : verifying that the Flutter app and FastAPI backend, and the backend and its connected services (OpenAI, Google Cloud Vision), work correctly together end to end.
- **Usability testing** : verifying that the voice-first flow can be completed relying solely on audio feedback, without reference to the screen.
- **Performance / response-time testing** : observing the responsiveness of on-device object detection, cloud-dependent OCR, transcription, and web search.

5.2.1 Testing Tools and Technologies

The following tools and approaches were used:

- Flutter test framework and manual UI testing for screen navigation and widget behavior.
- Android emulator and a physical Android device for camera, microphone, and wake-word testing.
- FastAPI's automatically generated API documentation (Swagger/OpenAPI interface) for interactive endpoint testing.

- Postman and curl for direct, controlled testing of backend endpoints.
- Camera-based manual testing for object detection and OCR accuracy under varying lighting and distance.
- Direct testing of the Google Cloud Vision OCR integration through the /read-text endpoint.
- Direct testing of the YOLO/TFLite object detection models (general COCO model and custom indoor model) using the live camera feed.
- Direct testing of the MiDaS depth estimation service through the /depth/midas endpoint.
- Manual testing of OpenAI API responses (routing, scene answers, OCR summarization, web search) for relevance and consistency.
- Backend console/log inspection for tracing request flow and diagnosing failures.

5.2.2 System Functions Tested

Testing covered Lumin’s main functional areas: onboarding and profile setup; voice-first home navigation and wake-word activation; object detection combining a general COCO-based model with a custom Roboflow-trained indoor model; MiDaS-based depth and proximity estimation; OCR-based text reading with conversational follow-up questions; web search with cited sources; speech transcription; text-to-speech output; the FastAPI backend endpoints; and error handling across all of the above.

5.3 Test Cases and Results

Table 1 presents the detailed test cases executed against Lumin. The majority of test cases passed as expected; two limitations were observed and are discussed further in Section 5.5.5. Table 1.a lists the scenario and input for each test, and Table 1.b lists the corresponding expected result, actual result, and status.

Table 1.a Detailed Test Cases (Scenario and Input)

Test Case ID	Requirement / Feature	Test Description	Input / Action
TC-01	Onboarding / First Launch	App is launched for the first time on a device	Fresh install, no prior setup data
TC-02	Permission Handling	Camera and microphone permissions are requested during setup	User accepts both permission prompts

Test Case ID	Requirement / Feature	Test Description	Input / Action
TC-03	Profile Setup	User enters name, email, and emergency contact during onboarding	Sample profile data entered
TC-04	Returning User Flow	App is relaunched after setup was already completed	Existing installation with completed setup
TC-05	Wake-Word Activation	Wake word "Lumin" is spoken to launch the assistant	Spoken wake word
TC-06	Voice Routing: Object Detection	Spoken command requests object/scene information	"What is in front of me?"
TC-07	Voice Routing: OCR	Spoken command requests text reading	"Read this document"
TC-08	Voice Routing: Web Search	Spoken command requests external knowledge	"Who is Albert Einstein?"
TC-09	Ambiguous Command Handling	Spoken command is unclear or insufficiently specific	A vague or incomplete voice command
TC-10	Object Detection COCO Model	General object model detects a common object in view	Camera pointed at object
TC-11	Object Detection Custom Indoor Model	Custom Roboflow-trained model detects an indoor household object	Camera pointed at object
TC-12	Scene Description	Detected objects are converted into a spoken scene summary	Multiple objects detected in one frame
TC-13	Depth / Proximity Estimation	MiDaS depth estimation labels object proximity	Objects at varying distances from the camera
TC-14	Scene Question Answering	User asks a question about the detected scene	"Is there a chair near me?" sent to /ask
TC-15	OCR: English Text	Camera captures a clearly printed English page and sends it to /read-text	Well-lit image of printed English text
TC-16	OCR: Arabic Text	Camera captures printed Arabic text	Well-lit image of printed Arabic text
TC-17	OCR: Degraded Conditions	Text is captured under poor lighting or blur	Dim or low-contrast image of printed text
TC-18	OCR: No Text Present	Image contains no readable text	Photo of a blank wall or plain object

Test Case ID	Requirement / Feature	Test Description	Input / Action
TC-19	OCR Follow-up (Context Sufficient)	User asks a follow-up question answerable from captured text	“What is the expiry date?” sent to /read-text/question
TC-20	OCR Follow-up (Web Fallback)	Follow-up question requires information beyond captured text	“What is this product used for?” referencing a label with no such detail
TC-21	Web Search with Citations	User asks a question requiring external/current information	Spoken general-knowledge question
TC-22	Text-to-Speech Output	Assistant response is converted to speech	Text response returned from /ask or /web-search
TC-23	Speech Transcription: Noisy Input	User speech is transcribed under background noise	Spoken question recorded in a moderately noisy environment
TC-24	Backend Health Check	Health endpoint is queried	GET request to /health
TC-25	Invalid Input Handling	Malformed base64 image or audio payload is submitted	Corrupted base64 string sent to a backend endpoint

Table 1.b Detailed Test Cases (Results and Status, matched by Test Case ID)

Test Case ID	Expected Result	Actual Result	Status
TC-01	Onboarding/setup screen appears requesting permissions and profile details	As expected	Passed
TC-02	App proceeds to Home screen with camera and microphone access enabled	As expected	Passed
TC-03	Profile data is saved, and setup is marked as complete	As expected	Passed
TC-04	App skips onboarding and opens directly to the Home screen	As expected	Passed
TC-05	Foreground service detects the wake word and opens the app / starts listening	As expected	Passed
TC-06	/route-home-command selects open_object_detection, and the app navigates accordingly	As expected	Passed

Test Case ID	Expected Result	Actual Result	Status
TC-07	/route-home-command selects open_ocr and the OCR screen opens	As expected	Passed
TC-08	/route-home-command selects open_web_search and the Web Search screen opens	As expected	Passed
TC-09	Backend returns a clarify action, and the app asks a short clarification question	As expected	Passed
TC-10	Object is detected with a bounding box, label, and confidence score	As expected	Passed
TC-11	Custom-class object is detected and merged with general model output	As expected	Passed
TC-12	App verbally summarizes nearby objects with position information	As expected	Passed
TC-13	Each object receives a proximity label (VERY CLOSE, CLOSE, MID, or FAR)	As expected	Passed
TC-14	Assistant returns a relevant spoken answer based on detected objects	As expected	Passed
TC-15	Google Cloud Vision extracts the text and the app reads it aloud accurately	As expected	Passed
TC-16	Extracted Arabic text is preserved and read aloud correctly	As expected	Passed
TC-17	Extracted text should remain reasonably accurate	OCR accuracy decreased noticeably under low-light and blurred conditions	Partial Limitation observed /
TC-18	App reports that no text was detected instead of returning an error	As expected	Passed
TC-19	Router determines OCR context is sufficient and answers directly	As expected	Passed
TC-20	Router falls back to web search and returns a combined answer	As expected	Passed
TC-21	/web-search returns a summarized spoken answer with source citations shown as buttons	As expected	Passed
TC-22	OpenAI TTS (/tts-openai) voices the response; Edge TTS (/tts-edge) used when the primary service is unavailable	As expected	Passed

Test Case ID	Expected Result	Actual Result	Status
TC-23	Transcription should remain reasonably accurate	Transcription accuracy declined with significant background noise	Partial Limitation observed /
TC-24	Backend returns HTTP 200 with a healthy status and configured providers	As expected	Passed
TC-25	Backend rejects the request with a descriptive validation error	As expected	Passed

5.3.1 Screenshots of Tested Features

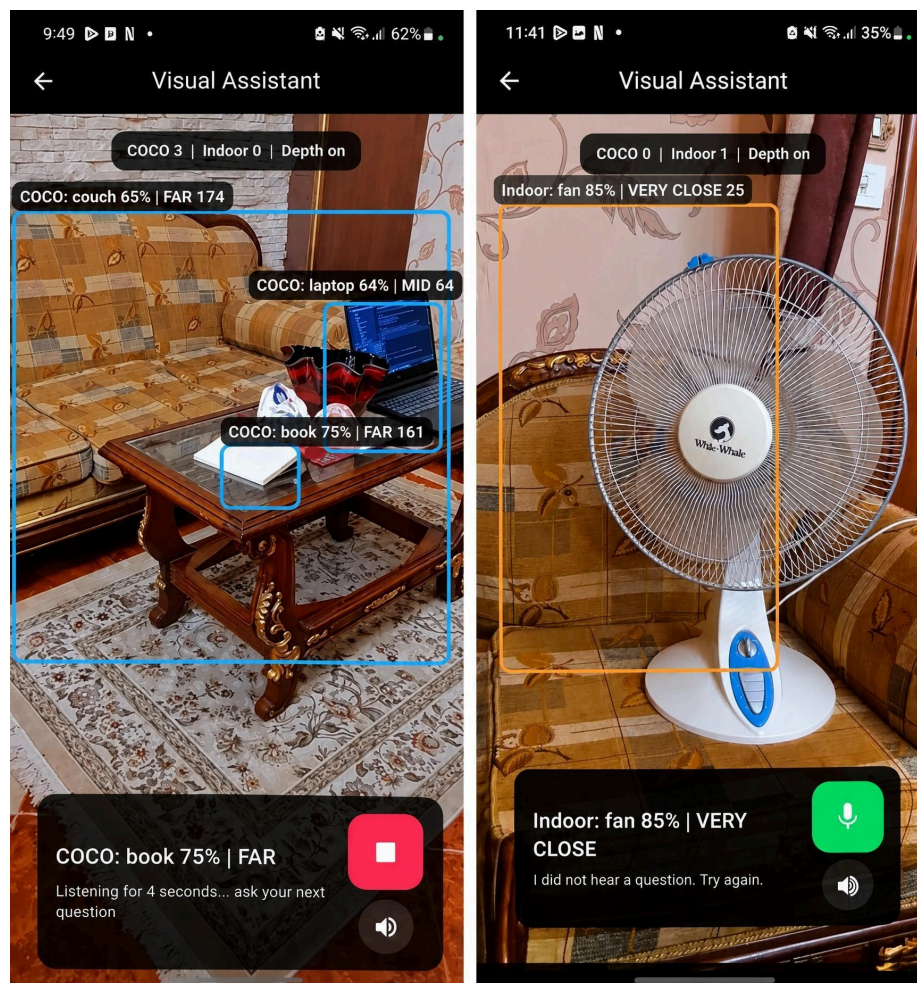


Fig. 9 Object Detection screen: general COCO-based model (left) vs. custom indoor model detecting the added “fan” class (right)



Fig. 10 OCR screen: text extracted from a medicine package label (left) and a printed book page (right)



Fig. 11 OCR follow-up web-search fallback: side-effect question asked in the OCR screen (bottom), and the backend log showing the router detecting insufficient OCR context and calling web_search before answering (top)

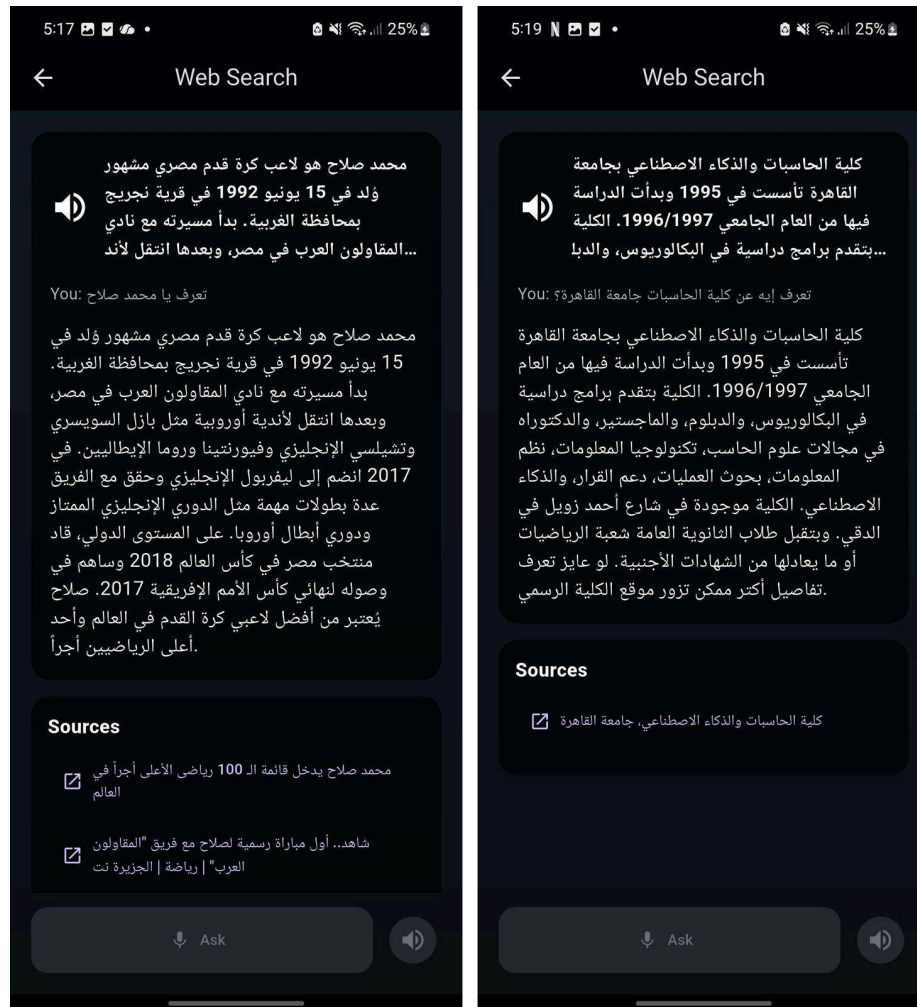


Fig. 12 Web Search screen: two example queries, each with a cited spoken answer

5.4 Test Results by System Requirement

Table 2 aggregates the test case results by major system requirement, providing a higher-level view of how each core capability performed during testing.

Table 2 Test Results by Major System Requirement

System Requirement	Related Test Cases	Result Summary
Voice-first navigation and command routing	TC-05 to TC-09	Wake-word activation and tool-calling-based routing performed reliably across the tested commands
Object detection and scene description	TC-10 to TC-14	General and custom indoor models produced accurate detections; scene summaries were coherent and useful

System Requirement	Related Test Cases	Result Summary
Depth and proximity estimation	TC-13	Proximity labels were consistent with the physical layout of test scenes
OCR text reading and follow-up Q&A	TC-15 to TC-20	OCR performed reliably under good lighting; accuracy decreased under poor lighting or blur
Web search with citations	TC-21	Search answers were relevant and included working source citations
Speech transcription and text-to-speech	TC-22, TC-23	TTS output (OpenAI/Edge) was clear; transcription accuracy decreased in noisy conditions
Backend reliability and error handling	TC-24, TC-25	Health checks and input validation all behaved as expected

23 of 25 test cases passed (92%). The remaining 2 test cases: OCR under degraded lighting/blur conditions (TC-17) and speech transcription under noisy input (TC-23) showed partial results, as discussed in Section 5.5.5.

5.5 Project Evaluation

This section evaluates Lumin beyond individual test outcomes, considering cost, performance relative to comparable assistive systems, and broader environmental and social/political implications.

5.5.1 Cost

Lumin was developed using a standard smartphone and open-source software tools, keeping development cost primarily limited to developer time. Table 3 summarizes the main cost categories associated with the system.

Table 3 Cost Evaluation

Cost Category	Description
Development cost	Primarily developer time; Flutter and FastAPI are open-source, so no framework licensing fees were incurred
Cloud API usage	Recurring, usage-based cost from OpenAI (transcription, reasoning, web search, TTS) and Google Cloud Vision (OCR), billed per request/token
Device cost	No dedicated hardware is required; Lumin runs on the user's existing Android smartphone

Cost Category	Description
Hosting	During development the backend ran on a local developer machine; a production deployment would add cloud hosting cost
Open-source components	Flutter, FastAPI, TFLite runtime, and the YOLO model family are free to use, reducing overall project cost

Compared to dedicated assistive hardware devices, Lumin’s reliance on a standard smartphone keeps the overall cost of ownership lower, since no specialized equipment needs to be purchased; the main ongoing cost is cloud API usage, which scales with how often the app is used.

Exact real-world running cost depends heavily on usage volume and on which OpenAI model tier is selected, but current published pricing (mid-2026) gives a reasonable estimate. Google Cloud Vision’s Text Detection (OCR) feature is billed per unit after a free monthly allowance, and new Google Cloud accounts additionally receive a one-time free trial credit. This matches the project’s own experience: after roughly 140 OCR requests, both the monthly free allotment and the trial credit remained almost untouched, so the OCR cost incurred so far has been effectively \$0.

OpenAI’s per-request cost is harder to pin to a single figure because it depends on which model handles routing, scene/OCR question answering, transcription, and text-to-speech, and how many tokens each call consumes. Table 4 summarizes representative rates for each service Lumin depends on, alongside the project’s own measured usage.

Table 4 Estimated API Usage Cost

Service / Feature	Pricing (as of mid-2026)	Notes / Project Usage
Google Cloud Vision Text Detection (OCR)	\$1.50 per 1,000 units after the first 1,000 units used each month, which are free; new accounts also receive \$300 in free trial credit valid for 90 days	About 140 OCR requests made during testing; \$0 spent so far, well within the monthly free tier and the trial credit
OpenAI routing, scene/OCR answers, web search summarisation	Billed per token; cost depends on which model tier is selected (lower-cost “mini” models are well suited to short routing/classification calls, larger models to longer answers)	Powers command routing, scene questions, OCR follow-up answers, and web-search summaries
OpenAI speech transcription	\$0.006 per minute of audio	Transcribes spoken commands and questions

Service / Feature	Pricing (as of mid-2026)	Notes / Project Usage
OpenAI Text-to-Speech (tts-1)	\$15 per 1,000,000 characters (about \$0.015 per 1,000 characters)	Converts assistant replies to speech; Edge TTS used as a free fallback when unavailable
Combined OpenAI usage (all of the above)	Usage-based; no fixed monthly fee	About \$5 spent to date while still returning a large number of requests, consistent with per-call costs of a small fraction of a cent for short routing calls up to a few cents for longer answers

Because both providers price per request or per token rather than a fixed subscription, cost scales directly with how many people use Lumin and how often. At current testing volumes, the project’s combined spend about \$5 to OpenAI and effectively \$0 to Google Cloud Vision shows that development and testing can be run at negligible cost. A production deployment serving many concurrent users would need to budget more carefully for OpenAI usage in particular, since it is billed per token rather than per request and larger models or longer responses cost proportionally more.

5.5.2 Time Compared to Contemporary Systems

Lumin’s object detection runs on-device using TFLite, giving near real-time results for scene understanding without depending on network latency. OCR, web search, and assistant question-answering depend on the FastAPI backend and external AI services, and are therefore subject to network and processing latency. Compared with relying on manual human assistance, Lumin provides substantially faster access to text, object, and web-based information. Compared with fully offline assistive systems, Lumin’s online features can take longer to respond but offer richer, more contextual answers, since they draw on large language models and live web search rather than fixed, pre-programmed responses.

Lumin’s cloud-dependent features were measured for response latency during testing. OCR requests averaged 2.94 seconds, transcription averaged 2.27 seconds (with occasional outliers under noisy input, as noted in Section 5.5.5), and web search averaged 6.49 seconds. Object detection is not included in this comparison because it runs through the on-device TFLite pipeline rather than the external network round trip that governs OCR, transcription, and web search response times. Its responsiveness depends on different factors in on-device inference, so it is not directly comparable to the network-bound latencies discussed in this section.

5.5.3 Environmental Impact

Lumin's environmental footprint is comparatively low because it runs entirely on a user's existing smartphone rather than requiring dedicated, single-purpose hardware, avoiding additional electronic waste. By allowing users to have documents and signs read aloud rather than requiring printed or braille alternatives, the app can also modestly reduce paper dependence in some scenarios. The main environmental cost comes from the server-side energy consumed by cloud AI usage: OpenAI and Google Cloud Vision requests and the smartphone's own energy consumption, both of which are minor relative to the footprint of dedicated hardware devices.

5.5.4 Social and Political Impact

Socially, Lumin supports independence and accessibility for visually impaired users by helping them recognize objects, understand their surroundings, read printed text, and access web-based information without depending on another person being present. This contributes to inclusive access to information and can support day-to-day tasks such as reading signs, labels, and documents.

From a political and ethical standpoint, Lumin's reliance on live camera images and voice recordings raises privacy considerations, since captured images may include other people or sensitive text, and voice recordings may include personal information. Consent, transparent data handling, and secure management of API keys and captured media are treated as necessary safeguards. Lumin is also intended to complement, not replace, human support in situations that involve real physical danger, since the system provides relative, voice-based guidance rather than guaranteed safety assurances.

5.5.5 Limitations Observed During Testing

Testing surfaced two notable limitations:

- OCR accuracy decreased under poor lighting conditions and with blurred or low-contrast images, occasionally producing incomplete text extraction.
- Speech transcription accuracy was affected by background noise, occasionally leading to misrecognized words in the transcribed command or question.

Both limitations depend on external, uncontrollable conditions (ambient lighting and ambient noise) rather than on defects in the application logic itself, and represent reasonable directions for future improvement, such as image pre-processing for low-light OCR or noise suppression for speech input.

5.6 Chapter Summary

Testing showed that Lumin's major functions- wake-word activation, voice-based routing, object detection with depth-based proximity estimation, OCR-based text reading with follow-up question answering, web search with citations, and text-to-speech output worked successfully across the scenarios evaluated. The backend's API endpoints responded correctly, including under invalid-input and network-failure conditions. Overall system performance depends on external factors such as camera quality, lighting conditions, background noise, and the quality of the available internet connection, since several of Lumin's most capable features rely on cloud-based AI services. These dependencies do not undermine the system's core reliability but do represent realistic constraints for a mobile assistive application and are consistent with limitations commonly reported in comparable assistive vision systems.

Chapter Six

Conclusion And Future Work

6.1 Conclusion

This project presented Lumin, an intelligent voice-first smartphone application designed to improve the independence, accessibility, and environmental awareness of blind and low-vision users. The system combines recent advances in artificial intelligence, computer vision, speech technologies, and large language models to provide an interactive assistive experience without requiring specialized hardware or pre-built indoor maps.

Lumin integrates multiple artificial intelligence technologies within a unified architecture. Object detection is performed using both a general-purpose YOLO26 model and a custom fine-tuned YOLO26 model trained on indoor objects commonly found in Egyptian homes. Relative depth estimation using the MiDaS model provides proximity information that helps users better understand the spatial relationship between surrounding objects. The system also incorporates Optical Character Recognition (OCR) using Google Cloud Vision, enabling users to read documents, labels, and signs. Unlike conventional OCR systems, Lumin integrates OCR with a web agent, allowing users to ask follow-up questions about previously extracted text and automatically perform web searches whenever additional information is required.

The application follows a hybrid client-server architecture that balances real-time performance and computational efficiency. Lightweight operations such as camera access, wake-word detection, audio recording, and object detection are executed locally on the Android device, while computationally intensive tasks including speech transcription, OCR reasoning, prompt orchestration, web search, language generation, and text-to-speech are securely handled by the FastAPI backend through external AI services. This architecture protects sensitive API credentials while maintaining responsive interaction.

One of the major contributions of this work is the implementation of intelligent tool calling. Rather than requiring users to manually select application features, it analyzes each spoken request, determines the user's intent, and automatically routes the request to the appropriate module, including object detection, OCR, or web search. This creates a seamless conversational experience and significantly reduces the cognitive effort required to interact with the application.

Another important contribution is the integration of OCR with the web agent. Instead of functioning as a simple text reader, the OCR module becomes an interactive assistant capable of understanding context, answering follow-up questions, and combining recognized text with external knowledge whenever necessary. This context-aware approach provides a more natural and accessible reading experience for visually impaired users.

Overall, the project successfully achieved its primary objectives by developing a functional prototype capable of providing intelligent object detection, relative depth estimation, conversational OCR, AI-powered web search, voice-first interaction, and context-aware assistance through a unified conversational interface. The resulting system demonstrates how modern artificial intelligence can be integrated into an accessible smartphone application that supports greater independence and improves the daily lives of visually impaired users.

6.2 Future Work

Although Lumin provides a practical and effective assistive solution, several enhancements can further improve its capabilities in future versions.

Personalized User Experience

Future versions of Lumin can provide a more personalized user experience by allowing users to customize the assistant according to their individual preferences. For example, users could select their preferred voice, speaking speed, response length, language, and interaction style. Such personalization would improve accessibility and user comfort while allowing the assistant to better adapt to different users and their needs.

Navigation Assistance Enhancement

During the development of LUMIN, a navigation assistance prototype was implemented and evaluated using object detection, relative depth estimation, and prompt-based reasoning to determine whether the path ahead was obstructed or unobstructed and to generate appropriate spoken guidance for the user. Experimental testing demonstrated encouraging results in detecting obstacles and recognizing clear pathways.

However, the evaluation also showed that the response speed depended heavily on the text-to-speech implementation. The locally executed text-to-speech engine provided sufficiently fast spoken feedback for real-time obstacle awareness, whereas the OpenAI text-to-speech service introduced noticeable latency due to remote speech generation and network communication. Future work can improve this functionality by integrating low-latency streaming speech synthesis or more natural on-device text-to-speech models that combine high speech quality with the fast response required for real-time navigation assistance

.

Expanding the Training Dataset

The object detection component can be further improved by expanding the training dataset with a larger variety of indoor environments, including offices, hospitals, universities, shopping malls, airports, and public buildings. Increasing the number of object categories and continuously retraining the custom YOLO26 model would improve robustness, generalization, and detection accuracy across diverse real-world environments.

Resources

International Agency for the Prevention of Blindness (IAPB) Vision Atlas (2020):

<https://visionatlas.iapb.org/country-data/egypt/>

Microsoft Seeing AI: <https://www.microsoft.com/en-us/garage/wall-of-fame/seeing-ai/>

Google Lookout: <https://support.google.com/accessibility/android/answer/9031274?hl=en>

Be My Eyes: <https://www.bemyeyes.com/>

ScribeMe: <https://scribeme-ai.com/>

Roboflow Dataset: <https://app.roboflow.com/lumin-frm5s/last-v-17/1>

YOLO26 Documentation: <https://docs.ultralytics.com/models/yolo26>

Google Cloud Vision: <https://docs.cloud.google.com/vision/docs/drag-and-drop>